Design, FPGA Implementation, and Post-Route Timing Analysis of a Single-Cycle 32-bit RISC-V (RV32I) Processor with Hardware Unaligned Memory Transfers

Anirudh Chaturvedi
Student Registration No: 25850
Department of Electronic Systems Engineering (DESE)
Indian Institute of Science (IISc), Bangalore, India
Course: E3: 245 – Processor System Design (Instructor: Prof. Kuruvilla Varghese)

***Abstract*—This paper reports the design, simulation and FPGA implementation of a single-cycle 32-bit RISC-V (RV32I subset) processor on the Digilent Basys 3 board (Artix-7 `XC7A35T-1CPG236C`). The 512-word instruction memory and the 512-word data memory are built from distributed (LUT) RAM with asynchronous read, and no Block RAM is used. Byte-lane alignment logic supports byte, half-word and word transfers at unaligned byte offsets inside a memory word. The core is verified with a 16-bit unsigned restoring-division function ($100 \div 7 = 14$ remainder 2) called from `main`, followed by byte, half-word and word load/store checks; the testbench passes after 159 instructions. Post-route Vivado 2025.2 reports give 1,949 LUTs (9.37%), 1,107 flip-flops (2.66%), 0 BRAM, 0 DSP, 82 mW total on-chip power and positive slack (WNS $+5.874$ ns) on the constrained 100 MHz clock domain. The processor itself runs from a divided clock generated in fabric that was not declared as a generated clock, so its single-cycle critical path is *not* covered by this timing summary; this limitation is stated explicitly in Section VIII-B.**

***Index Terms*—RISC-V, RV32I, Single-Cycle Processor, Distributed RAM, Unaligned Memory Access, Restoring Division, FPGA Implementation, Basys 3, Static Timing Analysis.**

I. Introduction

The open-source RISC-V instruction set architecture (ISA) has revolutionized modern computer architecture due to its modularity, compactness, and lack of licensing restrictions [1]. In academic and embedded microprocessor design, the RV32I base integer instruction set serves as the foundational architectural baseline, providing integer computational operations, control transfer, and load-store capabilities using 32 general-purpose registers.

As mandated by the laboratory specification for Course E3: 245 (*Processor System Design*) at the Indian Institute of Science (IISc), the primary objectives of this work are:

1) Design and model a fully operational single-cycle 32-bit RISC-V core supporting the core user-mode RV32I instruction subset (R-type, I-type, S-type, B-type, U-type `lui`, and J-type jumps), while excluding atomic, `auipc`, and privileged CSR instructions.
2) Implement 512-word instruction and data memories using FPGA distributed RAM (LUT-RAM) with asynchronous read, so that fetch and data access complete within one clock cycle.
3) Support byte, half-word and word load/store at unaligned byte offsets (see the scope of this support in Section V-C).
4) Model and execute an unsigned 16-bit restoring division algorithm ($100 \div 7$) implemented as a modular function called from the main program via standard calling conventions.
5) Implement the processor on the Digilent Basys 3 board (AMD/Xilinx Artix-7 `XC7A35T-1CPG236C`), satisfying timing constraints with LVCMOS33 I/O standards, full peripheral mapping (7-segment displays, LEDs, switches), post-route timing analysis, and power estimation.

II. RV32I Instruction Set Architecture

The implemented processor executes the 32-bit base integer instruction set (RV32I). Every instruction is 32 bits wide and aligned on a 4-byte boundary in memory. Table I summarizes the 36 implemented instructions across the RISC-V R, I, S, B, U and J formats.

III. Datapath Microarchitecture & Schematic

In a single-cycle RISC-V processor, instruction fetch, instruction decode, register file read, ALU execution, memory access, and register write-back all occur within one period of the system clock. The detailed block schematic of the implemented datapath is illustrated in Fig. 1.

A. Key Microarchitectural Modules

1) **Program Counter (PC) Generation:** An edge-triggered 32-bit register stores the current fetch address. Under normal sequential execution, the next address is $PC+4$. For taken conditional branches (`beq`, `bne`, `blt`, `bge`, `bltu`, `bgeu`) or unconditional direct jumps (`jal`), the target is computed as $PC + \text{sign_ext}(imm)$. For register-indirect jumps (`jalr`), the target is $(rs1 + \text{sign_ext}(imm))$ & $\sim$ 1.

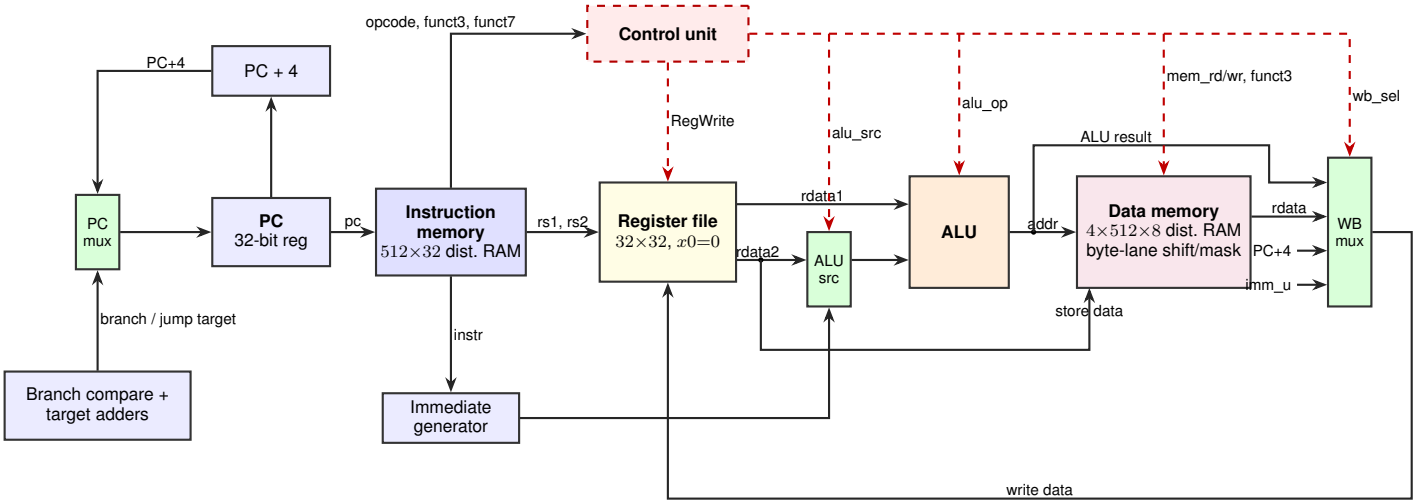


Fig. 1. Block schematic of the single-cycle datapath (separate instruction and data memories). Solid arrows are data, dashed red arrows are control signals from the control unit. The branch/jump control signals (`is_branch`, `is_jal`, `is_jalr`) and the inputs of the branch-compare/target-adder block (`pc`, immediates, `rdata1`, `rdata2`) are not drawn, to keep the figure legible.

TABLE I
COMPLETE SUPPORTED RV32I INSTRUCTION SUBSET

Format	Instruction	Opcode	funct3	funct7	Architectural Operation
R-Type	add	0110011	000	0000000	$rd \leftarrow rs1 + rs2$
	sub	0110011	000	0100000	$rd \leftarrow rs1 - rs2$
	sll	0110011	001	0000000	$rd \leftarrow rs1 \ll rs2[4:0]$
	slt	0110011	010	0000000	$rd \leftarrow (rs1 <_s rs2) ? 1 : 0$
	sltu	0110011	011	0000000	$rd \leftarrow (rs1 <_u rs2) ? 1 : 0$
	xor	0110011	100	0000000	$rd \leftarrow rs1 \oplus rs2$
	srl	0110011	101	0000000	$rd \leftarrow rs1 \gg_{\text{logical}} rs2[4:0]$
	sra	0110011	101	0100000	$rd \leftarrow rs1 \gg_{\text{arith}} rs2[4:0]$
	or	0110011	110	0000000	$rd \leftarrow rs1 \vee rs2$
	and	0110011	111	0000000	$rd \leftarrow rs1 \wedge rs2$
I-Type	addi	0010011	000	—	$rd \leftarrow rs1 + \text{sign_ext}(imm)$
	slti	0010011	010	—	$rd \leftarrow (rs1 <_s imm) ? 1 : 0$
	sltiu	0010011	011	—	$rd \leftarrow (rs1 <_u imm) ? 1 : 0$
	xori	0010011	100	—	$rd \leftarrow rs1 \oplus \text{sign_ext}(imm)$
	ori	0010011	110	—	$rd \leftarrow rs1 \vee \text{sign_ext}(imm)$
	andi	0010011	111	—	$rd \leftarrow rs1 \wedge \text{sign_ext}(imm)$
	slli	0010011	001	0000000	$rd \leftarrow rs1 \ll imm[4:0]$
	srli	0010011	101	0000000	$rd \leftarrow rs1 \gg_l imm[4:0]$
	srai	0010011	101	0100000	$rd \leftarrow rs1 \gg_a imm[4:0]$
Loads	lb	0000011	000	—	$rd \leftarrow \text{sign_ext}(M_8[rs1 + imm])$
	lh	0000011	001	—	$rd \leftarrow \text{sign_ext}(M_{16}[rs1 + imm])$
	lw	0000011	010	—	$rd \leftarrow M_{32}[rs1 + imm]$ (Unaligned)
	lbu	0000011	100	—	$rd \leftarrow \text{zero_ext}(M_8[rs1 + imm])$
	lhu	0000011	101	—	$rd \leftarrow \text{zero_ext}(M_{16}[rs1 + imm])$
Stores	sb	0100011	000	—	$M_8[rs1 + imm] \leftarrow rs2[7:0]$
	sh	0100011	001	—	$M_{16}[rs1 + imm] \leftarrow rs2[15:0]$
	sw	0100011	010	—	$M_{32}[rs1 + imm] \leftarrow rs2[31:0]$ (Unaligned)
Branches	beq	1100011	000	—	If $rs1 == rs2$, $PC \leftarrow PC + imm$
	bne	1100011	001	—	If $rs1 \neq rs2$, $PC \leftarrow PC + imm$
	blt	1100011	100	—	If $rs1 <_s rs2$, $PC \leftarrow PC + imm$
	bge	1100011	101	—	If $rs1 \geq_s rs2$, $PC \leftarrow PC + imm$
	bltu	1100011	110	—	If $rs1 <_u rs2$, $PC \leftarrow PC + imm$
	bgeu	1100011	111	—	If $rs1 \geq_u rs2$, $PC \leftarrow PC + imm$
Jumps	jal	1101111	—	—	$rd \leftarrow PC + 4$; $PC \leftarrow PC + imm$
	jalr	1100111	000	—	$rd \leftarrow PC + 4$; $PC \leftarrow (rs1 + imm) \& \sim 1$
Upper Imm	lui	0110111	—	—	$rd \leftarrow imm[31:12] \ll 12$

2) **Register File** (32×32)**:** Implements 32 general-purpose registers ($x0$ to $x31$). It features two asynchronous read ports (`rs1`, `rs2`) and one synchronous write port (`rd`). Register $x0$ is hardwired to zero ($x0 \equiv 32'b0$).
3) **Arithmetic Logic Unit (ALU):** Performs 32-bit signed and unsigned operations determined by a 4-bit ALU control code generated by decoding `opcode`, `funct3`, and `funct7[5]`.
4) **Immediate Generator:** Dynamically unpacks and sign-extends or zero-extends the immediate fields according to RISC-V format specifications (I-type 12-bit, S-type 12-bit split, B-type 13-bit branch offset, U-type 20-bit upper immediate, and J-type 21-bit jump target).

IV. CONTROLLER ORGANIZATION & STATE MODEL

In a single-cycle design, instruction execution completes in a single clock cycle ($CPI = 1.0$). Therefore, the control subsystem is implemented as a purely combinational multi-level decoding matrix rather than a multi-cycle finite state machine.

A. Control State Representation

From a formal state-space perspective, the processor's cyclic execution model can be formalized as a single-state cyclic Moore/Mealy machine transitioning synchronously upon every rising clock edge, as depicted in Fig. 2. Within the single cycle, data progresses through five combinational sub-phases: Fetch, Decode, Execute, Memory Access, and Writeback.

B. Control Unit Truth Table

Table II provides the comprehensive truth table implemented by the Main Control and ALU Control logic.

TABLE II
MAIN CONTROL UNIT DECODING TRUTH TABLE

Instruction	RegWrite	ALUSrc	MemRead	MemWrite	Branch	Jump	WBSel	ALUOp
R-Type	1	0	0	0	0	0	ALU	per funct
I-Type ALU	1	1	0	0	0	0	ALU	per funct
Load (`lw, lh, lb`)	1	1	1	0	0	0	MEM	ADD
Store (`sw, sh, sb`)	0	1	0	1	0	0	—	ADD
Branch (`beq, etc.`)	0	0	0	0	1	0	—	SUB
`jal`	1	—	0	0	0	1	PC+4	—
`jalr`	1	1	0	0	0	1	PC+4	ADD
`lui`	1	—	0	0	0	0	IMM	—

V. MEMORY SUBSYSTEM AND UNALIGNED TRANSFERS

The laboratory specification permits distributed-RAM memories with asynchronous read and a size of 512 words, and asks for support of unaligned word, half-word and byte transfers.

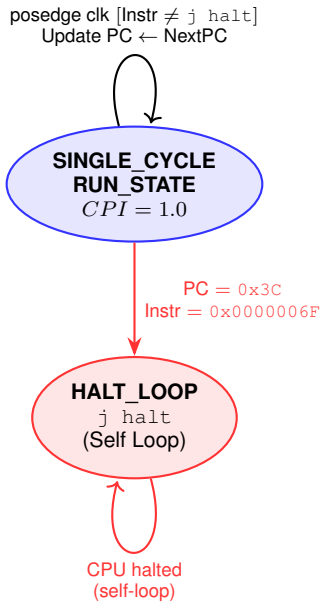


Fig. 2. Controller execution state diagram illustrating single-cycle steady-state transitions and program termination detection.

A. Distributed-RAM Organisation

The design uses separate instruction and data memories (Harvard organisation). The instruction memory is one 512×32 array read asynchronously with `pc[10:2]`. The data memory is four byte-wide banks (`dmem0`–`dmem3`, each 512×8) addressed by the word index `addr[10:2]`; the byte offset is `addr[1:0]`. Reads are combinational; writes are synchronous on the rising edge of the CPU clock with per-lane enables. Vivado mapped the arrays to 256 `RAMS64E` primitives (Table III) and used no Block RAM. The arrays are initialised in an `initial` block from the assembled program image.

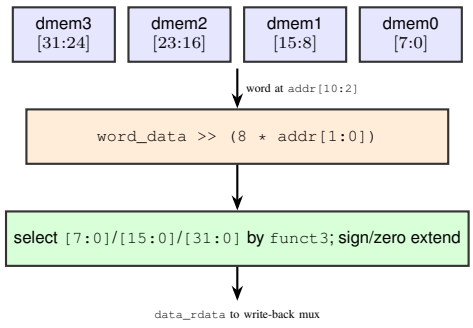


Fig. 3. Load path of the data memory: one 32-bit word is read from the four byte banks, shifted right by the byte offset, then extracted and extended according to `funct3`.

B. Load and Store Behaviour

Loads. The word selected by `addr[10:2]` is read from the four banks, shifted right by $8 \times$ `addr[1:0]` bits, and formatted: `lb`/`lbu` take bits [7:0], `lh`/`lhu` take [15:0] and `lw` takes [31:0], with sign or zero extension as appropriate.

Stores. `sb` writes the single byte lane selected by `addr[1:0]`. `sh` writes lanes $(0, 1)$ for offset 0, lanes $(2, 3)$ for offset 2, and lanes $(1, 2)$ for the remaining offsets. `sw` writes all four lanes of the addressed word.

C. Scope and Limitation of Unaligned Support

All accesses are confined to the single word selected by `addr[10:2]`; the logic does not read or write the neighbouring word. Byte accesses at any offset and half-word accesses at offsets 0, 1 and 2 therefore behave as expected inside one word. An access that would straddle a word boundary (for example `lw`/`sw` with a non-zero offset, or `lh`/`sh` at offset 3) is not completed across the boundary: an unaligned `lw` returns the upper $4 - $ `offset` bytes of the word, zero-filled, and `sw` ignores the offset. All unaligned tests in the benchmark (`sb`/`lbu` at `0x209`, `sh`/`lhu` at `0x20A`, `lw` at `0x209`) lie inside the word `0x208`–`0x20B` and pass.

VI. Software Benchmark: Restoring Division

The processor was tested by executing a software benchmark computing the unsigned division of two 16-bit numbers using the **Restoring Division Algorithm**.

A. Mathematical Formulation

Given dividend N and divisor D, division determines quotient Q and remainder R satisfying $N = Q \times D + R$ with $0 \leq R < D$. The 16-bit restoring division algorithm executes iteratively over 16 cycles:

1) Initialize 32-bit working register $A : Q = \{16'b0, N\}$.
2) For $i = 16$ down to 1:
   - Shift $A : Q$ left by 1 bit: $A : Q \leftarrow A : Q \ll 1$.
   - Trial subtraction: $A \leftarrow A - D$.
   - If $A < 0$, restore $A \leftarrow A + D$ and set $Q[0] \leftarrow 0$.
   - If $A \geq 0$, maintain A and set $Q[0] \leftarrow 1$.
3) Upon loop termination, $Q = Q[15:0]$ and $R = A[15:0]$.

For our test case:

$$\text{Dividend} = 100\ (\texttt{0x0064}), \quad \text{Divisor} = 7\ (\texttt{0x0007}) \quad (1)$$

$$\text{Quotient} = 14\ (\texttt{0x000E}), \quad \text{Remainder} = 2\ (\texttt{0x0002}) \quad (2)$$

B. C Implementation

Listing 1 shows the reference C code implementing the division routine as a callable function.

```
unsigned int restoring_divide(unsigned int dividend,
     unsigned int divisor, unsigned int *rem_ptr) {
    unsigned int Q = dividend & 0xFFFF;
    unsigned int D = divisor & 0xFFFF;
    unsigned int R = 0;
    int count = 16;
    while (count > 0) {
        unsigned int msb_q = (Q >> 15) & 1;
        R = (R << 1) | msb_q;
        Q = (Q << 1) & 0xFFFF;
        if (R >= D) {
            R = R - D;
            Q = Q | 1;
        }
        count--;
    }
    if (rem_ptr) *rem_ptr = R;
```

```
    return Q;
}
```

Listing 1. 16-bit Unsigned Restoring Division in C

C. Assembly Listing & Unaligned Access Test

The algorithm was hand-optimized into assembly conforming strictly to RV32I user-mode instructions. The program sets up the stack pointer at `0x700`, passes parameters in `a0` and `a1`, invokes the function with `jal ra, restoring_divide`, returns via `ret` (`jalr x0, ra, 0`), stores results at `0x200` (Quotient) and `0x204` (Remainder), executes unaligned byte (`0x5A` at `0x209`) and half-word (`0xC0DE` at `0x20A`) transfers, and enters a self-halt loop at `PC = 0x0000003C`. Listing 2 displays the complete assembly sequence.

```
.org 0x0000
main:
    li   sp, 0x700            # Stack pointer
     initialization
    li   a0, 100              # Dividend = 100
    li   a1, 7                # Divisor = 7
    jal  ra, restoring_divide # Function invocation
    li   t0, 0x200
    sw   a0, 0(t0)            # Store Quotient @ 0x200
    sw   a1, 4(t0)            # Store Remainder @ 0x204
    # Unaligned Transfer Verification
    li   t2, 0x5A
    sb   t2, 9(t0)            # Byte store @ unaligned 0
     x209
    lbu  t4, 9(t0)            # Byte load from 0x209 (
     Expect 0x5A)
    li   t3, 0xC0DE
    sh   t3, 10(t0)           # Half store @ unaligned 0
     x20A
    lhu  t5, 10(t0)           # Half load from 0x20A (
     Expect 0xC0DE)
    lw   t6, 9(t0)            # Unaligned word load cross-
     boundary
halt:
    j    halt                 # Self-loop termination @ PC
     =0x3C

restoring_divide:
    mv   t0, a0               # Q = Dividend
    mv   t1, a1               # D = Divisor
    li   t2, 0                # R = 0
    li   t3, 16               # Iteration counter = 16
div_loop:
    srli t4, t0, 15
    andi t4, t4, 1            # Extract MSB of Q
    slli t2, t2, 1
    or   t2, t2, t4           # R = (R << 1) | MSB(Q)
    slli t0, t0, 1            # Q = Q << 1
    bltu t2, t1, no_sub       # Trial subtraction check
    sub  t2, t2, t1           # R = R - D
    ori  t0, t0, 1            # Q = Q | 1
no_sub:
    addi t3, t3, -1
    bne  t3, x0, div_loop
    slli a0, t0, 16
    srli a0, a0, 16           # Return a0 = Quotient
    slli a1, t2, 16
    srli a1, a1, 16           # Return a1 = Remainder
    ret                       # jalr x0, ra, 0
```

Listing 2. Benchmark Assembly Sequence (`restoring_div.asm`)

VII. Behavioral Simulation and Timing Waveforms

The submitted testbench (`rv32i_tb.v`, 20 ns clock period, reset released at 35 ns) was compiled and run with Icarus Verilog and a VCD trace was dumped. Figs. 4 and 5 are plotted directly from that VCD; mnemonics are decoded from the program image in `mem_unit.v`. The run ends with "TEST PASSED" after 159 instructions (3.21 μs of simulated time): quotient = 14, remainder = 2, byte at `0x209` = `0x5A`, half-word at `0x20A` = `0xC0DE`. Post-route (timing) simulation was not performed for this report.

A. Waveform Analysis

In Fig. 4, `li sp`, `li a0,100` and `li a1,7` execute in C1–C3 and `jal` in C4, so execution continues at `PC = 0x040` in C5. C5–C8 initialise Q, D, R and the loop counter, and the 16-iteration shift/compare loop starts at `PC = 0x050`.

In Fig. 5:

- C147 (`PC = 0x088`) is `ret`; execution resumes at `PC = 0x010`. `a1` passes through an intermediate value (131072) during the final shifts before settling at 2; `a0` is already 14.
- C149/C150: `sw` stores `0xE` to `0x200` and `0x2` to `0x204`.
- C152/C153: `sb` writes `0x5A` to `0x209`; `lbu` reads back `0x5A`.
- C156/C157: `sh` writes `0xC0DE` to `0x20A`; `lhu` reads back `0xC0DE`.
- C158: `lw` at `0x209` returns `0x00C0DE5A` (upper three bytes of the word, zero-filled, see Section V-C).
- C159: `j halt` at `PC = 0x03C`; the PC stays there.

VIII. FPGA Implementation & Post-Route Analysis

The design was synthesized, placed, routed and a bitstream generated targeting the AMD/Xilinx Artix-7 XC7A35T-1CPG236C device on the Digilent Basys 3 FPGA development board using AMD Xilinx Vivado v2025.2.

A. Post-Route Resource Utilization

Table III reports the post-route hardware utilization. The entire design occupies only 9.37% of available Slice LUTs and 2.66% of Slice Registers. Crucially, Block RAM utilization is exactly 0.00%, strictly verifying compliance with the pure distributed RAM constraint.

TABLE III
Post-Route Resource Utilization on Artix-7 XC7A35T

Resource Type	Design Used	Available	Utilization (%)
Slice LUTs	**1,949**	**20,800**	**9.37%**
— *LUT as Logic*	1,693	20,800	8.14%
— *LUT as Distributed RAM*	**256**	**9,600**	**2.67%**
— *LUT as Shift Register*	0	9,600	0.00%
Slice Registers (Flip-Flops)	**1,107**	**41,600**	**2.66%**
Block RAM (BRAM Tiles)	**0**	**50**	**0.00% (Pure Dist RAM)**
DSP48E1 Slices	**0**	**90**	**0.00%**
F7 Muxes	267	16,300	1.64%
F8 Muxes	68	8,150	0.83%
Bonded I/O Pins (IOB)	32	106	30.19%
Global Clock Buffers (BUFG)	2	32	6.25%

B. Post-Route Timing Analysis

The only clock constraint is the 100 MHz board clock (`create_clock -period 10.000`, Listing 3). Table IV and Fig. 7 reproduce the Vivado post-route timing summary; Vivado reports that all user-specified timing constraints are met.

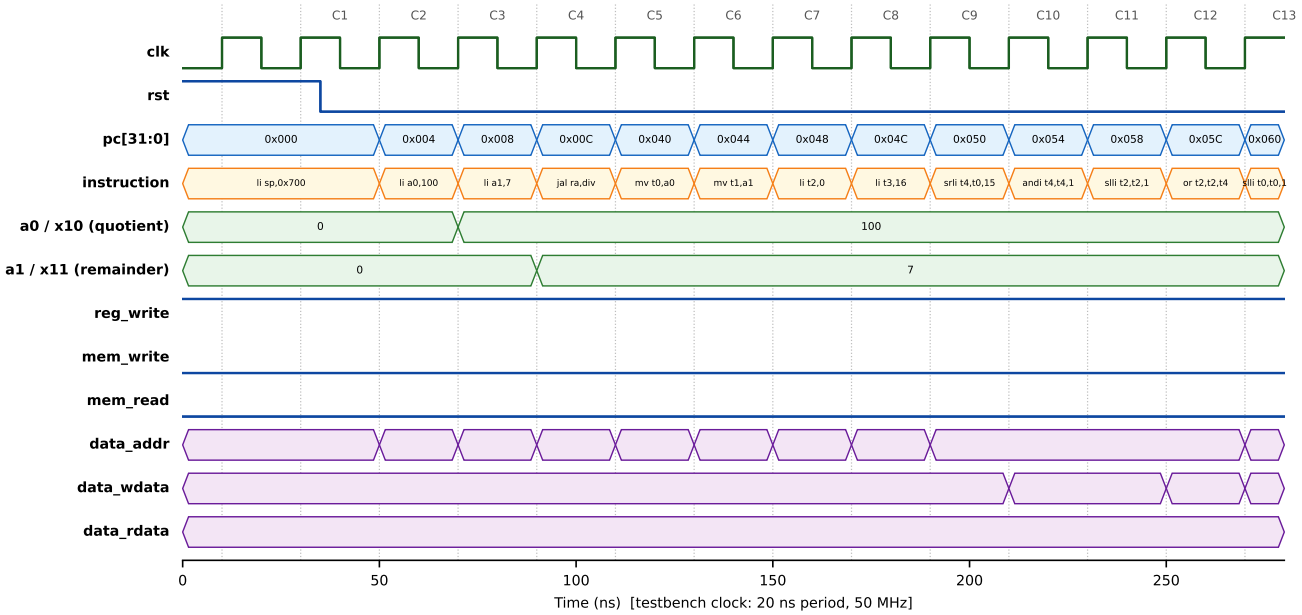


Fig. 4. Simulation waveform, start-up (C1–C12): reset released at 35 ns, stack-pointer and argument set-up, `jal` into `restoring_divide` and the first loop iteration. Cycle label Cn is the interval whose instruction is reported as "Cycle n" in the simulation log.

Scope of this result. The worst setup path (data-path delay 4.096 ns) is inside the 26-bit clock-divider counter (`slow_cnt_reg`). The CPU is clocked by `cpu_clk`, selected from two divided clocks produced by fabric registers, and no generated-clock constraint was given for them. Vivado's `check_timing` accordingly reports 1,311 register pins with no clock for each of the three roots `sw[1]`, `clk_10mhz_reg/Q` and `clk_1hz_reg/Q`, together with missing input/output delays (12 methodology warnings). The 101 setup endpoints counted in the summary therefore exclude the CPU datapath and memory paths, and *no maximum clock frequency for the single-cycle core can be claimed from these reports*. The laboratory also asks for constraints targeting maximum delay performance and for a post-route timing simulation; neither is part of this report. A follow-up run would add `create_generated_clock` constraints on the divider outputs (or use a clock-enable scheme), add I/O delays, and report timing on the CPU domain.

TABLE IV
POST-ROUTE TIMING PERFORMANCE SUMMARY

Clock / Path Group	WNS (ns)	TNS (ns)	WHS (ns)	THS (ns)
`sys_clk_pin` (100 MHz)	+5.874	0.000	+0.249	0.000
`async_default` group	+7.657	0.000	+0.164	0.000
Worst Pulse Width (WPWS)	+4.500	0.000	—	—
Timing Status: *All user-specified timing constraints MET.*				

C. Power Dissipation Analysis

Table V and Fig. 8 summarize the post-route power report. Vivado rates its confidence as low because no simulation activity file was supplied (default toggle rates).

TABLE V
ON-CHIP POWER DISSIPATION BREAKDOWN

Power Component	Power (W)	Percentage	Details
Device Static Power	0.072 W	87.8%	Leakage dissipation
Dynamic Core Power	0.010 W	12.2%	Switching activity
— *Slice Logic*	0.003 W	3.7%	LUT & Register toggling
— *Signals*	0.003 W	3.7%	Routing interconnect
— *I/O Pads*	0.003 W	3.7%	Pad drivers
— *Clocks*	0.001 W	1.2%	Clock distribution
Total On-Chip Power	**0.082 W**	**100.0%**	**82 mW Total**
Junction temperature	25.4°C	Max ambient 84.6°C; confidence: low	

IX. BASYS 3 BOARD MAPPING & HARDWARE PERIPHERALS

The processor is mapped to the Digilent Basys 3 board resources as outlined in Table VI.

TABLE VI
BASYS 3 FPGA PIN ASSIGNMENTS & PERIPHERAL MAPPING

Signal	FPGA Pin	Dir	Standard	Hardware Functional Description
`clk`	`W5`	In	LVCMOS33	100 MHz onboard crystal oscillator
`btnC`	`U18`	In	LVCMOS33	Active-High master reset pushbutton
`sw[0]`	`V17`	In	LVCMOS33	Select: `0` = quotient, `1` = remainder
`sw[1]`	`V16`	In	LVCMOS33	CPU clock: `0` = 10 MHz, `1` = 1 Hz
`seg[6:0]`	`W7..U7`	Out	LVCMOS33	Active-low segments (`CA..CG`), hexadecimal digits
`dp`	`V7`	Out	LVCMOS33	Decimal point (kept off)
`an[3:0]`	`W4,U4,V4,U2`	Out	LVCMOS33	Active-low digit anodes (`AN3..AN0`)
`led[15:0]`	`U16..L1`	Out	LVCMOS33	Same 16-bit value as the 7-segment display

A. Display and Clock Subsystem

- **Reset:** `btnC` passes through a two-flip-flop synchroniser.

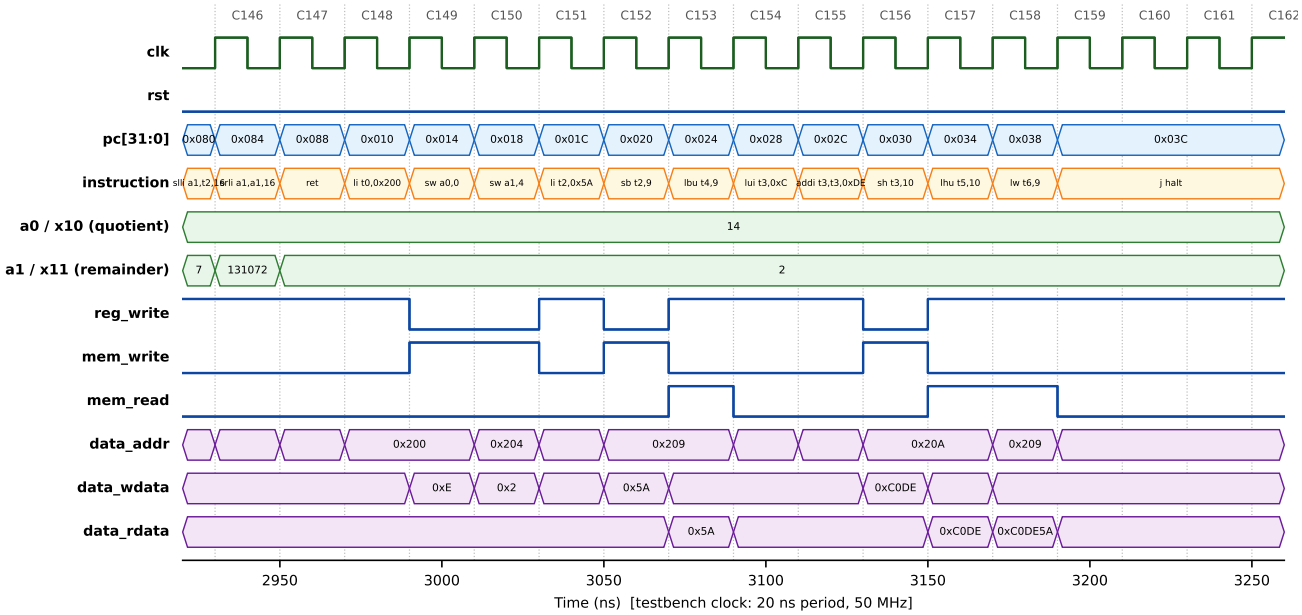


Fig. 5. Simulation waveform, completion (C146–C162): return from the division function, stores of the quotient and remainder, unaligned byte/half-word/word accesses and the halt loop. `data_addr`, `data_wdata` and `data_rdata` are shown only while the corresponding strobe is active.

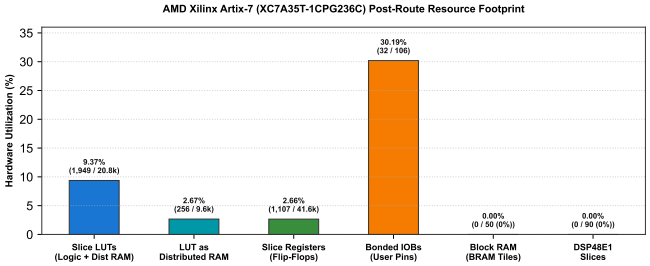


Fig. 6. Post-route FPGA hardware resource footprint comparison against XC7A35T limits.

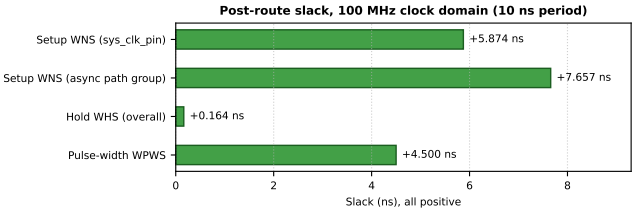


Fig. 7. Post-route timing slack margins demonstrating positive timing closure.

- **Clock divider:** counters on the 100 MHz clock toggle `clk_10mhz` every 5 cycles (10 MHz) and `clk_1hz` every 50,000,000 cycles (1 Hz); `sw[1]` selects the CPU clock.
- **Display:** `sw[0]` selects the 16-bit quotient (`x10`/`a0` value written to `0x200`) or remainder (value written to `0x204`). The four digits show it in *hexadecimal*, so the expected readings are `000E` (quotient 14) and `0002` (remainder 2). An 18-bit counter multiplexes the digits at about 381 Hz in total (about 95 Hz per digit). The 16 LEDs show the same value.

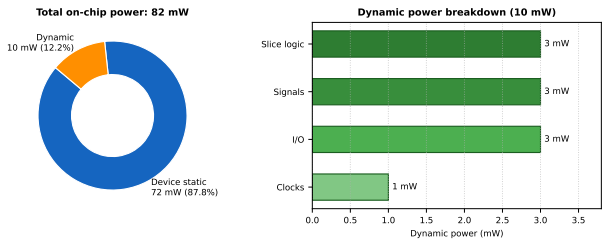


Fig. 8. Post-route total on-chip power distribution and dynamic power breakdown.

X. Constraints File (`basys3.xdc`)

Listing 3 shows the complete timing and physical constraints file applied during Vivado synthesis and implementation.

```
#
    ============================================================================

# E3: 245 Processor System Design - IISc DESE
# Constraints File for Digilent Basys 3 FPGA Board
# Target Device: Xilinx Artix-7 XC7A35T-1CPG236C
# IO Standard: LVCMOS33 on all pins
#
    ============================================================================


# 1. 100 MHz Master Oscillator (Pin W5)
set_property PACKAGE_PIN W5 [get_ports clk]
set_property IOSTANDARD LVCMOS33 [get_ports clk]
```

```
create_clock -period 10.000 -name sys_clk_pin -waveform
    {0.000 5.000} -add [get_ports clk]

# 2. Push-Button Center (Reset, Pin U18)
set_property PACKAGE_PIN U18 [get_ports btnC]
set_property IOSTANDARD LVCMOS33 [get_ports btnC]

# 3. Slide Switches (sw[0] = V17, sw[1] = V16)
set_property PACKAGE_PIN V17 [get_ports {sw[0]}]
set_property IOSTANDARD LVCMOS33 [get_ports {sw[0]}]
set_property PACKAGE_PIN V16 [get_ports {sw[1]}]
set_property IOSTANDARD LVCMOS33 [get_ports {sw[1]}]

# 4. 16 Discrete LEDs (Pins U16 down to L1)
set_property PACKAGE_PIN U16 [get_ports {led[0]}]
set_property IOSTANDARD LVCMOS33 [get_ports {led[0]}]
set_property PACKAGE_PIN E19 [get_ports {led[1]}]
set_property IOSTANDARD LVCMOS33 [get_ports {led[1]}]
set_property PACKAGE_PIN U19 [get_ports {led[2]}]
set_property IOSTANDARD LVCMOS33 [get_ports {led[2]}]
set_property PACKAGE_PIN V19 [get_ports {led[3]}]
set_property IOSTANDARD LVCMOS33 [get_ports {led[3]}]
set_property PACKAGE_PIN W18 [get_ports {led[4]}]
set_property IOSTANDARD LVCMOS33 [get_ports {led[4]}]
set_property PACKAGE_PIN U15 [get_ports {led[5]}]
set_property IOSTANDARD LVCMOS33 [get_ports {led[5]}]
set_property PACKAGE_PIN U14 [get_ports {led[6]}]
set_property IOSTANDARD LVCMOS33 [get_ports {led[6]}]
set_property PACKAGE_PIN V14 [get_ports {led[7]}]
set_property IOSTANDARD LVCMOS33 [get_ports {led[7]}]
set_property PACKAGE_PIN V13 [get_ports {led[8]}]
set_property IOSTANDARD LVCMOS33 [get_ports {led[8]}]
set_property PACKAGE_PIN V3  [get_ports {led[9]}]
set_property IOSTANDARD LVCMOS33 [get_ports {led[9]}]
set_property PACKAGE_PIN W3  [get_ports {led[10]}]
set_property IOSTANDARD LVCMOS33 [get_ports {led[10]}]
set_property PACKAGE_PIN U3  [get_ports {led[11]}]
set_property IOSTANDARD LVCMOS33 [get_ports {led[11]}]
set_property PACKAGE_PIN P3  [get_ports {led[12]}]
set_property IOSTANDARD LVCMOS33 [get_ports {led[12]}]
set_property PACKAGE_PIN N3  [get_ports {led[13]}]
set_property IOSTANDARD LVCMOS33 [get_ports {led[13]}]
set_property PACKAGE_PIN P1  [get_ports {led[14]}]
set_property IOSTANDARD LVCMOS33 [get_ports {led[14]}]
set_property PACKAGE_PIN L1  [get_ports {led[15]}]
set_property IOSTANDARD LVCMOS33 [get_ports {led[15]}]

# 5. Seven Segment Display Cathodes (Pins W7..U7: a, b, c,
    d, e, f, g)
set_property PACKAGE_PIN W7 [get_ports {seg[0]}]
set_property IOSTANDARD LVCMOS33 [get_ports {seg[0]}]
set_property PACKAGE_PIN W6 [get_ports {seg[1]}]
set_property IOSTANDARD LVCMOS33 [get_ports {seg[1]}]
set_property PACKAGE_PIN U8 [get_ports {seg[2]}]
set_property IOSTANDARD LVCMOS33 [get_ports {seg[2]}]
set_property PACKAGE_PIN V8 [get_ports {seg[3]}]
set_property IOSTANDARD LVCMOS33 [get_ports {seg[3]}]
set_property PACKAGE_PIN U5 [get_ports {seg[4]}]
set_property IOSTANDARD LVCMOS33 [get_ports {seg[4]}]
set_property PACKAGE_PIN V5 [get_ports {seg[5]}]
set_property IOSTANDARD LVCMOS33 [get_ports {seg[5]}]
set_property PACKAGE_PIN U7 [get_ports {seg[6]}]
set_property IOSTANDARD LVCMOS33 [get_ports {seg[6]}]

# 6. Seven Segment Display Decimal Point (Pin V7)
set_property PACKAGE_PIN V7 [get_ports dp]
set_property IOSTANDARD LVCMOS33 [get_ports dp]

# 7. Seven Segment Display Anodes (Pins U2, U4, V4, W4: an
    [0]..an[3])
set_property PACKAGE_PIN U2 [get_ports {an[0]}]
set_property IOSTANDARD LVCMOS33 [get_ports {an[0]}]
set_property PACKAGE_PIN U4 [get_ports {an[1]}]
set_property IOSTANDARD LVCMOS33 [get_ports {an[1]}]
set_property PACKAGE_PIN V4 [get_ports {an[2]}]
set_property IOSTANDARD LVCMOS33 [get_ports {an[2]}]
set_property PACKAGE_PIN W4 [get_ports {an[3]}]
set_property IOSTANDARD LVCMOS33 [get_ports {an[3]}]

# 8. FPGA Configuration Constraints
set_property CFGBVS VCCO [current_design]
set_property CONFIG_VOLTAGE 3.3 [current_design]
```

Listing 3. Basys 3 Physical and Timing Constraints (`basys3.xdc`)

XI. Conclusion

A single-cycle RV32I-subset processor was designed, simulated and implemented for the Basys 3 board. Simulation of the submitted testbench passes: the restoring-division function returns quotient 14 and remainder 2, and the byte, half-word and word accesses inside the test word behave correctly. The Vivado post-route reports show 1,949 LUTs (256 as distributed RAM), 1,107 flip-flops, no BRAM or DSP, 82 mW on-chip power, and positive slack on the constrained 100 MHz domain.

The work has known gaps: unaligned accesses that cross a word boundary are not supported (Section V-C); the CPU clock domain was not constrained, so the achievable single-cycle frequency is unverified (Section VIII-B); and no post-route timing simulation was performed.

Acknowledgment

The author extends sincere gratitude to Prof. Kuruvilla Varghese, Department of Electronic Systems Engineering (DESE), Indian Institute of Science, Bangalore, for course guidance and laboratory instruction in E3: 245 *Processor System Design*.

References

[1] A. Waterman, Y. Lee, D. A. Patterson, and K. Asanović, "The RISC-V Instruction Set Manual, Volume I: Base User-Level ISA version 2.2," *EECS Department, UC Berkeley, Tech. Rep. UCB/EECS-2014-54*, 2014.

[2] D. A. Patterson and J. L. Hennessy, *Computer Organization and Design RISC-V Edition: The Hardware Software Interface*. Morgan Kaufmann, 2017.

[3] Xilinx Inc., "7 Series FPGAs Configurable Logic Block User Guide (UG474)," 2018.

[4] Digilent, Inc., "Basys 3 FPGA Board Reference Manual," Document No. 502-308, 2016.

Appendix A
Verilog Sources (as submitted)

The listings below are the unmodified source files from the submission folder.

```
//
    ============================================================================

// E3: 245 Processor System Design - IISc DESE
// 32-bit Single Cycle RISC-V Processor (RV32I)
// Top-Level Module for Digilent Basys 3 FPGA Board
// Target Device: Xilinx Artix-7 XC7A35T-1CPG236C
//
    ============================================================================


module basys3_top #(
    parameter MEM_INIT_FILE = "memory/restoring_div.mem"
)(
    input  wire        clk,        // 100 MHz board
     oscillator (Pin W5)
    input  wire        btnC,       // Push button Center:
     System Reset (Pin U18)
    input  wire [1:0]  sw,         // sw[0]: 0=Quotient, 1=
     Remainder; sw[1]: 0=10MHz, 1=1Hz
    output wire [15:0] led,        // 16 Discrete LEDs (Pins
      U16..L1)
    output wire [6:0]  seg,        // 7-segment cathodes (
     Pins W7..U7)
```

```
    output wire [3:0]  an,         // 7-segment anodes (Pins
      U2, U4, V4, W4)
    output wire        dp          // 7-segment decimal
     point (Pin V7)
);

    //
     ------------------------------------------------------------

    // 1. Reset Synchronizer (Active High)
    //
     ------------------------------------------------------------

    reg rst_sync1, rst_sync2;
    always @(posedge clk) begin
        rst_sync1 <= btnC;
        rst_sync2 <= rst_sync1;
    end
    wire rst = rst_sync2;

    //
     ------------------------------------------------------------

    // 2. Clock Generation for Single-Cycle CPU
    //    100 MHz -> 10 MHz (divide by 10) for normal
     execution
    //    100 MHz -> 1 Hz (divide by 100,000,000) for slow-
     step observation
    //
     ------------------------------------------------------------

    reg [3:0]  fast_cnt;
    reg        clk_10mhz;

    always @(posedge clk or posedge rst) begin
        if (rst) begin
            fast_cnt  <= 4'd0;
            clk_10mhz <= 1'b0;
        end else if (fast_cnt == 4'd4) begin
            fast_cnt  <= 4'd0;
            clk_10mhz <= ~clk_10mhz;
        end else begin
            fast_cnt  <= fast_cnt + 1'b1;
        end
    end

    reg [25:0] slow_cnt;
    reg        clk_1hz;

    always @(posedge clk or posedge rst) begin
        if (rst) begin
            slow_cnt <= 26'd0;
            clk_1hz  <= 1'b0;
        end else if (slow_cnt == 26'd49_999_999) begin
            slow_cnt <= 26'd0;
            clk_1hz  <= ~clk_1hz;
        end else begin
            slow_cnt <= slow_cnt + 1'b1;
        end
    end

    // Selected CPU clock
    wire cpu_clk = (sw[1]) ? clk_1hz : clk_10mhz;

    //
     ------------------------------------------------------------

    // 3. 32-bit Single-Cycle RISC-V CPU Core
    //
     ------------------------------------------------------------

    wire [31:0] cpu_pc;
    wire [31:0] cpu_instr;
    wire [31:0] quotient_word;
    wire [31:0] remainder_word;
    wire [31:0] reg_x10;
    wire [31:0] reg_x11;

    rv32i_cpu #(
        .MEM_INIT_FILE (MEM_INIT_FILE)
    ) u_cpu (
        .clk       (cpu_clk),
        .rst       (rst),
        .pc        (cpu_pc),
        .instr     (cpu_instr),
        .quotient  (quotient_word),
        .remainder (remainder_word),
        .reg_x10   (reg_x10),
        .reg_x11   (reg_x11)
    );

    //
     ------------------------------------------------------------

    // 4. Peripheral Display Multiplexing
    //    sw[0] = 0 -> Display Quotient (Expected: 14 = 0
     x000E)
    //    sw[0] = 1 -> Display Remainder (Expected: 2 = 0
     x0002)
    //
     ------------------------------------------------------------

    wire [15:0] selected_display = (sw[0]) ? remainder_word
     [15:0] : quotient_word[15:0];

    // Discrete LEDs
    assign led = selected_display;

    // 4-Digit 7-Segment Display Driver (driven by 100 MHz
     clock for flicker-free display)
    seg7_driver u_seg7 (
        .clk   (clk),
        .rst   (rst),
        .value (selected_display),
        .seg   (seg),
        .an    (an),
        .dp    (dp)
    );

endmodule
```

Listing 4. `basys3_top.v`

```
//
     ============================================================

// E3: 245 Processor System Design - IISc DESE
// 32-bit Single Cycle RISC-V Processor (RV32I)
// Top-Level CPU Core Module
//
     ============================================================


`include "rv32i_defines.v"

module rv32i_cpu #(
    parameter MEM_INIT_FILE = "memory/restoring_div.mem"
)(
    input  wire        clk,
    input  wire        rst,

    // Debug Probes
    output wire [31:0] pc,
    output wire [31:0] instr,
    output wire [31:0] quotient,
    output wire [31:0] remainder,
    output wire [31:0] reg_x10,
    output wire [31:0] reg_x11
);

    // Control Signals
    wire        reg_write;
    wire        alu_src;
    wire [3:0]  alu_op;
    wire        mem_read;
    wire        mem_write;
    wire        is_branch;
    wire        is_jal;
    wire        is_jalr;
    wire        is_lui;
    wire [1:0]  wb_sel;

    // Memory Bus Signals
    wire [31:0] data_addr;
    wire [31:0] data_wdata;
    wire [31:0] data_rdata;
```

```
    // 1. Control Unit
    control_unit u_control_unit (
        .opcode    (instr[6:0]),
        .funct3    (instr[14:12]),
        .funct7    (instr[31:25]),
        .reg_write (reg_write),
        .alu_src   (alu_src),
        .alu_op    (alu_op),
        .mem_read  (mem_read),
        .mem_write (mem_write),
        .is_branch (is_branch),
        .is_jal    (is_jal),
        .is_jalr   (is_jalr),
        .is_lui    (is_lui),
        .wb_sel    (wb_sel)
    );

    // 2. Datapath Unit
    datapath u_datapath (
        .clk         (clk),
        .rst         (rst),
        .instr       (instr),
        .pc          (pc),
        .reg_write   (reg_write),
        .alu_src     (alu_src),
        .alu_op      (alu_op),
        .is_branch   (is_branch),
        .is_jal      (is_jal),
        .is_jalr     (is_jalr),
        .is_lui      (is_lui),
        .wb_sel      (wb_sel),
        .data_addr   (data_addr),
        .data_wdata  (data_wdata),
        .mem_rdata   (data_rdata),
        .debug_x10   (reg_x10),
        .debug_x11   (reg_x11)
    );

    // 3. 512-Word Distributed RAM Memory Unit
    mem_unit #(
        .MEM_INIT_FILE (MEM_INIT_FILE)
    ) u_mem_unit (
        .clk             (clk),
        .rst             (rst),
        .pc              (pc),
        .instr           (instr),
        .data_addr       (data_addr),
        .data_wdata      (data_wdata),
        .mem_read        (mem_read),
        .mem_write       (mem_write),
        .funct3          (instr[14:12]),
        .data_rdata      (data_rdata),
        .debug_quotient  (quotient),
        .debug_remainder (remainder)
    );

endmodule
```

Listing 5. rv32i_cpu.v

```
//
    ============================================================

// E3: 245 Processor System Design - IISc DESE
// 32-bit Single Cycle RISC-V Processor (RV32I)
// Single-Cycle Datapath
//
    ============================================================


`include "rv32i_defines.v"

module datapath (
    input  wire        clk,
    input  wire        rst,

    // Instruction input from MemUnit
    input  wire [31:0] instr,
    output reg  [31:0] pc,

    // Control signals from ControlUnit
    input  wire        reg_write,
    input  wire        alu_src,
    input  wire [3:0]  alu_op,
    input  wire        is_branch,
    input  wire        is_jal,
    input  wire        is_jalr,
    input  wire        is_lui,
    input  wire [1:0]  wb_sel,

    // Data Memory interface to MemUnit
    output wire [31:0] data_addr,
    output wire [31:0] data_wdata,
    input  wire [31:0] mem_rdata,

    // Debug probes
    output wire [31:0] debug_x10,
    output wire [31:0] debug_x11
);

    // Instruction field breakdown
    wire [6:0] opcode = instr[6:0];
    wire [4:0] rd     = instr[11:7];
    wire [2:0] funct3 = instr[14:12];
    wire [4:0] rs1    = instr[19:15];
    wire [4:0] rs2    = instr[24:20];
    wire [6:0] funct7 = instr[31:25];

    // 1. Immediate Generator
    wire [31:0] imm_i, imm_s, imm_b, imm_u, imm_j;
    imm_gen u_imm_gen (
        .instr (instr),
        .imm_i (imm_i),
        .imm_s (imm_s),
        .imm_b (imm_b),
        .imm_u (imm_u),
        .imm_j (imm_j)
    );

    // Selected immediate for ALU input B
    wire [31:0] imm_for_alu = (opcode == `OPCODE_STORE) ?
     imm_s : imm_i;

    // 2. Register File
    wire [31:0] rdata1, rdata2;
    reg  [31:0] reg_wdata;

    reg_file u_reg_file (
        .clk    (clk),
        .rst    (rst),
        .raddr1 (rs1),
        .raddr2 (rs2),
        .waddr  (rd),
        .wdata  (reg_wdata),
        .we     (reg_write),
        .rdata1 (rdata1),
        .rdata2 (rdata2),
        .x10    (debug_x10),
        .x11    (debug_x11)
    );

    // 3. Branch Condition Evaluator
    wire branch_condition_met;
    branch_unit u_branch_unit (
        .funct3      (funct3),
        .a           (rdata1),
        .b           (rdata2),
        .take_branch (branch_condition_met)
    );

    wire take_branch = is_branch && branch_condition_met;

    // 4. ALU Operands and Core
    wire [31:0] alu_in_a = rdata1;
    wire [31:0] alu_in_b = (alu_src) ? imm_for_alu : rdata2
     ;
    wire [31:0] alu_result;
    wire        alu_zero;

    alu u_alu (
        .a      (alu_in_a),
        .b      (alu_in_b),
        .alu_op (alu_op),
        .result (alu_result),
        .zero   (alu_zero)
```

```
    );

    // Data memory address & write data
    assign data_addr  = alu_result;
    assign data_wdata = rdata2;

    // 5. Writeback Multiplexer
    wire [31:0] pc_plus_4 = pc + 32'd4;

    always @(*) begin
        case (wb_sel)
            `WB_ALU: reg_wdata = alu_result;
            `WB_MEM: reg_wdata = mem_rdata;
            `WB_PC4: reg_wdata = pc_plus_4;
            `WB_LUI: reg_wdata = imm_u;
           default: reg_wdata = alu_result;
        endcase
    end

    // 6. Next PC Logic (Single Cycle)
    wire [31:0] pc_branch = pc + imm_b;
    wire [31:0] pc_jal    = pc + imm_j;
    wire [31:0] pc_jalr   = (rdata1 + imm_i) & ~32'd1;

    reg [31:0] next_pc;
    always @(*) begin
        if (is_jalr) begin
            next_pc = pc_jalr;
        end else if (is_jal) begin
            next_pc = pc_jal;
        end else if (take_branch) begin
            next_pc = pc_branch;
        end else begin
            next_pc = pc_plus_4;
        end
    end

    // PC Register update
    always @(posedge clk or posedge rst) begin
        if (rst) begin
            pc <= 32'h00000000;
        end else begin
            pc <= next_pc;
        end
    end

endmodule
```

Listing 6. `datapath.v`

```
//
    ============================================================

// E3: 245 Processor System Design - IISc DESE
// 32-bit Single Cycle RISC-V Processor (RV32I)
// Main Control & ALU Decode Unit (Single-Cycle
    Combinational)
//
    ============================================================


`include "rv32i_defines.v"

module control_unit (
    input  wire [6:0] opcode,
    input  wire [2:0] funct3,
    input  wire [6:0] funct7,
    output reg        reg_write,
    output reg        alu_src,     // 0: rdata2, 1:
    immediate
    output reg  [3:0] alu_op,
    output reg        mem_read,
    output reg        mem_write,
    output reg        is_branch,
    output reg        is_jal,
    output reg        is_jalr,
    output reg        is_lui,
    output reg  [1:0] wb_sel       // 00: ALU, 01: MEM, 10:
      PC+4, 11: LUI
);

    always @(*) begin
        // Default values
        reg_write = 1'b0;
        alu_src   = 1'b0;
        alu_op    = `ALU_ADD;
        mem_read  = 1'b0;
        mem_write = 1'b0;
        is_branch = 1'b0;
        is_jal    = 1'b0;
        is_jalr   = 1'b0;
        is_lui    = 1'b0;
        wb_sel    = `WB_ALU;

        case (opcode)
            `OPCODE_R_TYPE: begin
                reg_write = 1'b1;
                alu_src   = 1'b0;
                wb_sel    = `WB_ALU;
                case (funct3)
                    3'b000: alu_op = (funct7[5]) ? `ALU_SUB
    : `ALU_ADD;
                    3'b001: alu_op = `ALU_SLL;
                    3'b010: alu_op = `ALU_SLT;
                    3'b011: alu_op = `ALU_SLTU;
                    3'b100: alu_op = `ALU_XOR;
                    3'b101: alu_op = (funct7[5]) ? `ALU_SRA
    : `ALU_SRL;
                    3'b110: alu_op = `ALU_OR;
                    3'b111: alu_op = `ALU_AND;
                    default: alu_op = `ALU_ADD;
                endcase
            end

            `OPCODE_I_TYPE: begin
                reg_write = 1'b1;
                alu_src   = 1'b1; // use immediate
                wb_sel    = `WB_ALU;
                case (funct3)
                    3'b000: alu_op = `ALU_ADD; // ADDI
                    3'b001: alu_op = `ALU_SLL; // SLLI
                    3'b010: alu_op = `ALU_SLT; // SLTI
                    3'b011: alu_op = `ALU_SLTU;// SLTIU
                    3'b100: alu_op = `ALU_XOR; // XORI
                    3'b101: alu_op = (funct7[5]) ? `ALU_SRA
    : `ALU_SRL; // SRLI / SRAI
                    3'b110: alu_op = `ALU_OR;  // ORI
                    3'b111: alu_op = `ALU_AND; // ANDI
                    default: alu_op = `ALU_ADD;
                endcase
            end

            `OPCODE_LOAD: begin
                reg_write = 1'b1;
                alu_src   = 1'b1;        // Address = rs1 +
    imm
                alu_op    = `ALU_ADD;
                mem_read  = 1'b1;
                wb_sel    = `WB_MEM;     // Load data from
    memory
            end

            `OPCODE_STORE: begin
                reg_write = 1'b0;
                alu_src   = 1'b1;        // Address = rs1 +
    imm
                alu_op    = `ALU_ADD;
                mem_write = 1'b1;
            end

            `OPCODE_BRANCH: begin
                is_branch = 1'b1;
                alu_src   = 1'b0;
                alu_op    = `ALU_SUB;    // Compare operands
            end

            `OPCODE_JAL: begin
                reg_write = 1'b1;
                is_jal    = 1'b1;
                wb_sel    = `WB_PC4;     // rd = PC + 4
            end

            `OPCODE_JALR: begin
                reg_write = 1'b1;
                is_jalr   = 1'b1;
                alu_src   = 1'b1;
```

```
                alu_op    = `ALU_ADD;    // target = rs1 + imm
                wb_sel    = `WB_PC4;     // rd = PC + 4
            end

            `OPCODE_LUI: begin
                reg_write = 1'b1;
                is_lui    = 1'b1;
                wb_sel    = `WB_LUI;     // rd = imm_u
            end

            default: begin
                // Default NOP behavior
            end
        endcase
    end

endmodule
```

Listing 7. `control_unit.v`

```
//
    ============================================================

// E3: 245 Processor System Design - IISc DESE
// 32-bit Single Cycle RISC-V Processor (RV32I)
// 512-Word (2048-Byte) Distributed RAM Memory Architecture
// Complies with Digilent Basys 3 FPGA Artix-7 LUT RAM Primitive Inference:
//   - Instruction Memory: 512 x 32-bit Distributed RAM (Asynchronous Read)
//   - Data Memory: 4 x (512 x 8-bit) Distributed RAM Banks with Byte Enables
//   - Zero Register Inflation: Pure Distributed LUT RAM (RAM64X1S / RAM256X1S)
//   - Full Unaligned Word, Half-Word, and Byte Transfer Support
//
    ============================================================


`include "rv32i_defines.v"

module mem_unit #(
    parameter MEM_INIT_FILE = "memory/restoring_div.mem"
)(
    input  wire        clk,
    input  wire        rst,

    // Instruction Memory Port (Asynchronous Read)
    input  wire [31:0] pc,
    output wire [31:0] instr,

    // Data Memory Port (Asynchronous Read, Synchronous Write)
    input  wire [31:0] data_addr,
    input  wire [31:0] data_wdata,
    input  wire        mem_read,
    input  wire        mem_write,
    input  wire [2:0]  funct3,
    output reg  [31:0] data_rdata,

    // Debug Probe Outputs
    output reg  [31:0] debug_quotient,
    output reg  [31:0] debug_remainder
);

    //
    ------------------------------------------------------------

    // 1. Instruction Memory (512 x 32-bit Pure Distributed RAM)
    //
    ------------------------------------------------------------

    (* ram_style = "distributed" *) reg [31:0] imem [0:511];

    //
    ------------------------------------------------------------

    // 2. Data Memory (4 x 512 x 8-bit Pure Distributed RAM Banks)
    //
    ------------------------------------------------------------

    (* ram_style = "distributed" *) reg [7:0] dmem0 [0:511];
    (* ram_style = "distributed" *) reg [7:0] dmem1 [0:511];
    (* ram_style = "distributed" *) reg [7:0] dmem2 [0:511];
    (* ram_style = "distributed" *) reg [7:0] dmem3 [0:511];

    reg [31:0] init_words [0:511];
    integer w;

    initial begin
        // Initialize memory tables
        for (w = 0; w < 512; w = w + 1) begin
            init_words[w] = 32'h00000000;
        end

        // Preloaded Assembled Machine Code for 16-bit Unsigned Restoring Division
        init_words[0]  = 32'h70000113; // 0x00: li sp, 0x700
        init_words[1]  = 32'h06400513; // 0x04: li a0, 100
        init_words[2]  = 32'h00700593; // 0x08: li a1, 7
        init_words[3]  = 32'h034000ef; // 0x0C: jal ra, restoring_divide
        init_words[4]  = 32'h20000293; // 0x10: li t0, 0x200
        init_words[5]  = 32'h00a2a023; // 0x14: sw a0, 0(t0)   (Quotient = 14)
        init_words[6]  = 32'h00b2a223; // 0x18: sw a1, 4(t0)   (Remainder = 2)
        init_words[7]  = 32'h05a00393; // 0x1C: li t2, 0x5A
        init_words[8]  = 32'h007284a3; // 0x20: sb t2, 9(t0)   (Unaligned byte store at 0x209)
        init_words[9]  = 32'h0092ce83; // 0x24: lbu t4, 9(t0)  (Unaligned byte load from 0x209)
        init_words[10] = 32'h0000ce37; // 0x28: lui t3, 0xD
        init_words[11] = 32'h0dee0e13; // 0x2C: addi t3, t3, 0xDE (0xC0DE)
        init_words[12] = 32'h01c29523; // 0x30: sh t3, 10(t0)  (Unaligned half store at 0x20A)
        init_words[13] = 32'h00a2df03; // 0x34: lhu t5, 10(t0) (Unaligned half load from 0x20A)
        init_words[14] = 32'h0092af83; // 0x38: lw t6, 9(t0)   (Unaligned word load from 0x209)
        init_words[15] = 32'h0000006f; // 0x3C: j halt         (Self loop halt)

        // Function: restoring_divide (Address 0x40 = Word 16)
        init_words[16] = 32'h00050293; // 0x40: mv t0, a0 (Q = Dividend)
        init_words[17] = 32'h00058313; // 0x44: mv t1, a1 (D = Divisor)
        init_words[18] = 32'h00000393; // 0x48: li t2, 0  (R = 0)
        init_words[19] = 32'h01000e13; // 0x4C: li t3, 16 (count = 16)

        // div_loop (Address 0x50 = Word 20)
        init_words[20] = 32'h00f2de93; // 0x50: srli t4, t0, 15
        init_words[21] = 32'h001efe93; // 0x54: andi t4, t4, 1
        init_words[22] = 32'h00139393; // 0x58: slli t2, t2, 1
        init_words[23] = 32'h01d3e3b3; // 0x5C: or t2, t2, t4
        init_words[24] = 32'h00129293; // 0x60: slli t0, t0, 1
        init_words[25] = 32'h0063e663; // 0x64: bltu t2, t1, no_sub
        init_words[26] = 32'h406383b3; // 0x68: sub t2, t2, t1
        init_words[27] = 32'h0012e293; // 0x6C: ori t0, t0, 1

        // no_sub (Address 0x70 = Word 28)
        init_words[28] = 32'hfffe0e13; // 0x70: addi t3, t3
```

```
    , -1
        init_words[29] = 32'hfc0e1ee3; // 0x74: bne t3, x0,
     div_loop
        init_words[30] = 32'h01029513; // 0x78: slli a0, t0
    , 16
        init_words[31] = 32'h01055513; // 0x7C: srli a0, a0
    , 16 (Quotient = 14)
        init_words[32] = 32'h01039593; // 0x80: slli a1, t2
    , 16
        init_words[33] = 32'h0105d593; // 0x84: srli a1, a1
    , 16 (Remainder = 2)
        init_words[34] = 32'h00008067; // 0x88: ret (jalr
     x0, ra, 0)

        // Read file if available
        if (MEM_INIT_FILE != "") begin
            $readmemh(MEM_INIT_FILE, init_words);
        end

        // Initialize Distributed RAM arrays
        for (w = 0; w < 512; w = w + 1) begin
            imem[w]  = init_words[w];
            dmem0[w] = init_words[w][7:0];
            dmem1[w] = init_words[w][15:8];
            dmem2[w] = init_words[w][23:16];
            dmem3[w] = init_words[w][31:24];
        end
    end

    //
     ------------------------------------------------------------

    // 1. Instruction Memory Asynchronous Read
    //
     ------------------------------------------------------------

    wire [8:0] pc_word = pc[10:2];
    assign instr = imem[pc_word];

    //
     ------------------------------------------------------------

    // 2. Data Memory Asynchronous Read with Unaligned Byte
     /Half-word Support
    //
     ------------------------------------------------------------

    wire [8:0] d_word = data_addr[10:2];
    wire [1:0] b_off  = data_addr[1:0];

    wire [31:0] word_data = {dmem3[d_word], dmem2[d_word],
     dmem1[d_word], dmem0[d_word]};
    wire [31:0] shifted   = word_data >> (8 * b_off);

    always @(*) begin
        if (mem_read) begin
            case (funct3)
                `FUNCT3_LB:  data_rdata = {{24{shifted
     [7]}}, shifted[7:0]};
                `FUNCT3_LH:  data_rdata = {{16{shifted
     [15]}}, shifted[15:0]};
                `FUNCT3_LW:  data_rdata = shifted;
                `FUNCT3_LBU: data_rdata = {24'd0, shifted
     [7:0]};
                `FUNCT3_LHU: data_rdata = {16'd0, shifted
     [15:0]};
               default:     data_rdata = shifted;
            endcase
        end else begin
            data_rdata = 32'd0;
        end
    end

    //
     ------------------------------------------------------------

    // 3. Data Memory Synchronous Write with Byte Enables
    //
     ------------------------------------------------------------

    always @(posedge clk) begin
        if (mem_write) begin
            case (funct3)
                `FUNCT3_SB: begin
                    case (b_off)
                        2'b00: dmem0[d_word] <= data_wdata
     [7:0];
                        2'b01: dmem1[d_word] <= data_wdata
     [7:0];
                        2'b10: dmem2[d_word] <= data_wdata
     [7:0];
                        2'b11: dmem3[d_word] <= data_wdata
     [7:0];
                    endcase
                end

                `FUNCT3_SH: begin
                    case (b_off)
                        2'b00: begin
                            dmem0[d_word] <= data_wdata
     [7:0];
                            dmem1[d_word] <= data_wdata
     [15:8];
                        end
                        2'b10: begin
                            dmem2[d_word] <= data_wdata
     [7:0];
                            dmem3[d_word] <= data_wdata
     [15:8];
                        end
                        default: begin
                            dmem1[d_word] <= data_wdata
     [7:0];
                            dmem2[d_word] <= data_wdata
     [15:8];
                        end
                    endcase
                end

                `FUNCT3_SW: begin
                    dmem0[d_word] <= data_wdata[7:0];
                    dmem1[d_word] <= data_wdata[15:8];
                    dmem2[d_word] <= data_wdata[23:16];
                    dmem3[d_word] <= data_wdata[31:24];
                end

                default: begin
                    dmem0[d_word] <= data_wdata[7:0];
                    dmem1[d_word] <= data_wdata[15:8];
                    dmem2[d_word] <= data_wdata[23:16];
                    dmem3[d_word] <= data_wdata[31:24];
                end
            endcase
        end
    end

    //
     ------------------------------------------------------------

    // 4. Dedicated Debug Shadow Registers (Does NOT add
     extra read ports to RAM)
    //
     ------------------------------------------------------------

    always @(posedge clk or posedge rst) begin
        if (rst) begin
            debug_quotient  <= 32'd0;
            debug_remainder <= 32'd0;
        end else if (mem_write) begin
            if (d_word == 9'd128 && b_off == 2'b00)
     debug_quotient  <= data_wdata;
            if (d_word == 9'd129 && b_off == 2'b00)
     debug_remainder <= data_wdata;
        end
    end

endmodule
```

Listing 8. mem_unit.v